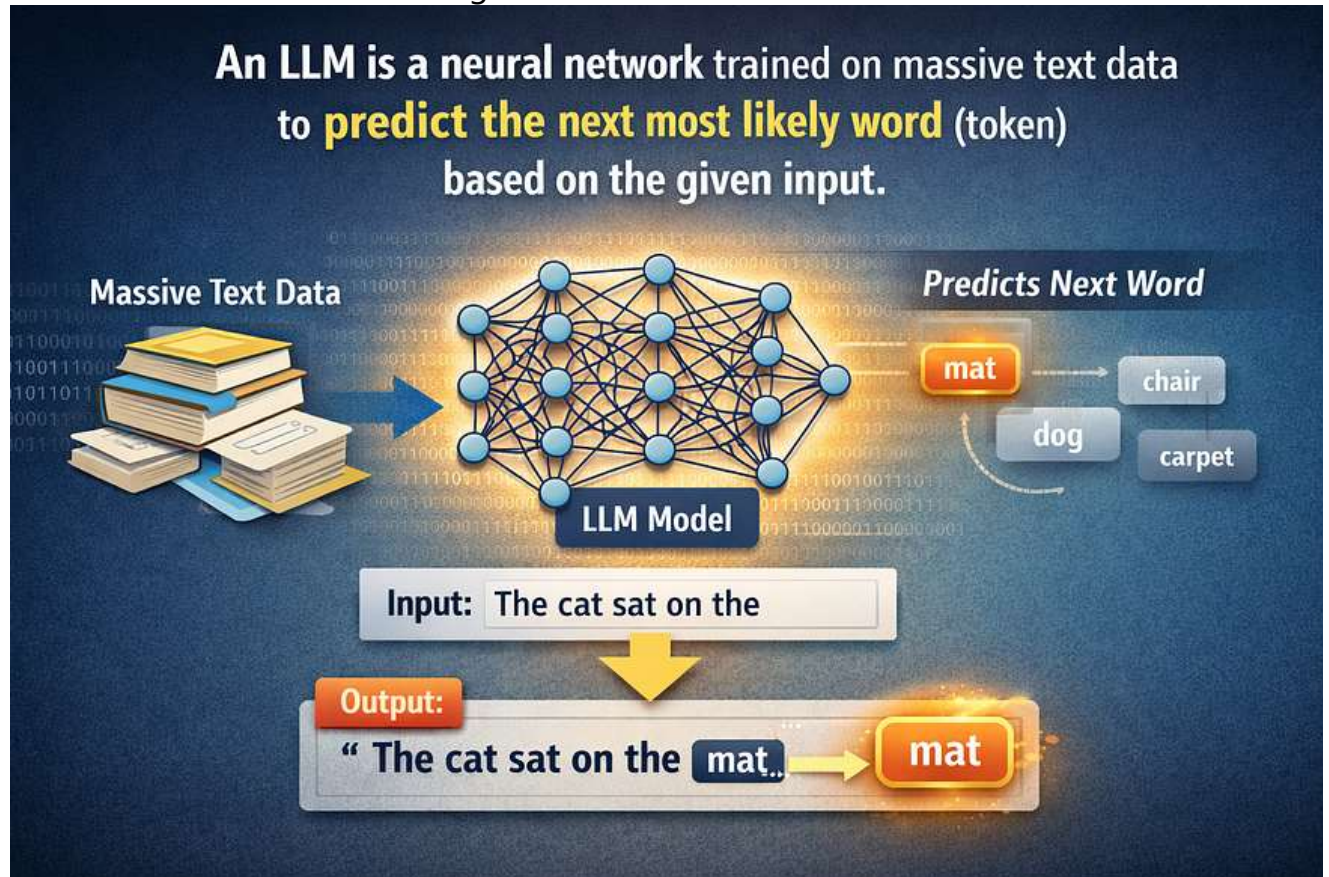


Understanding the building blocks behind modern AI models

Press enter or click to view image in full size



AI generated image

LLMs do NOT “know” answers. They predict what comes next.

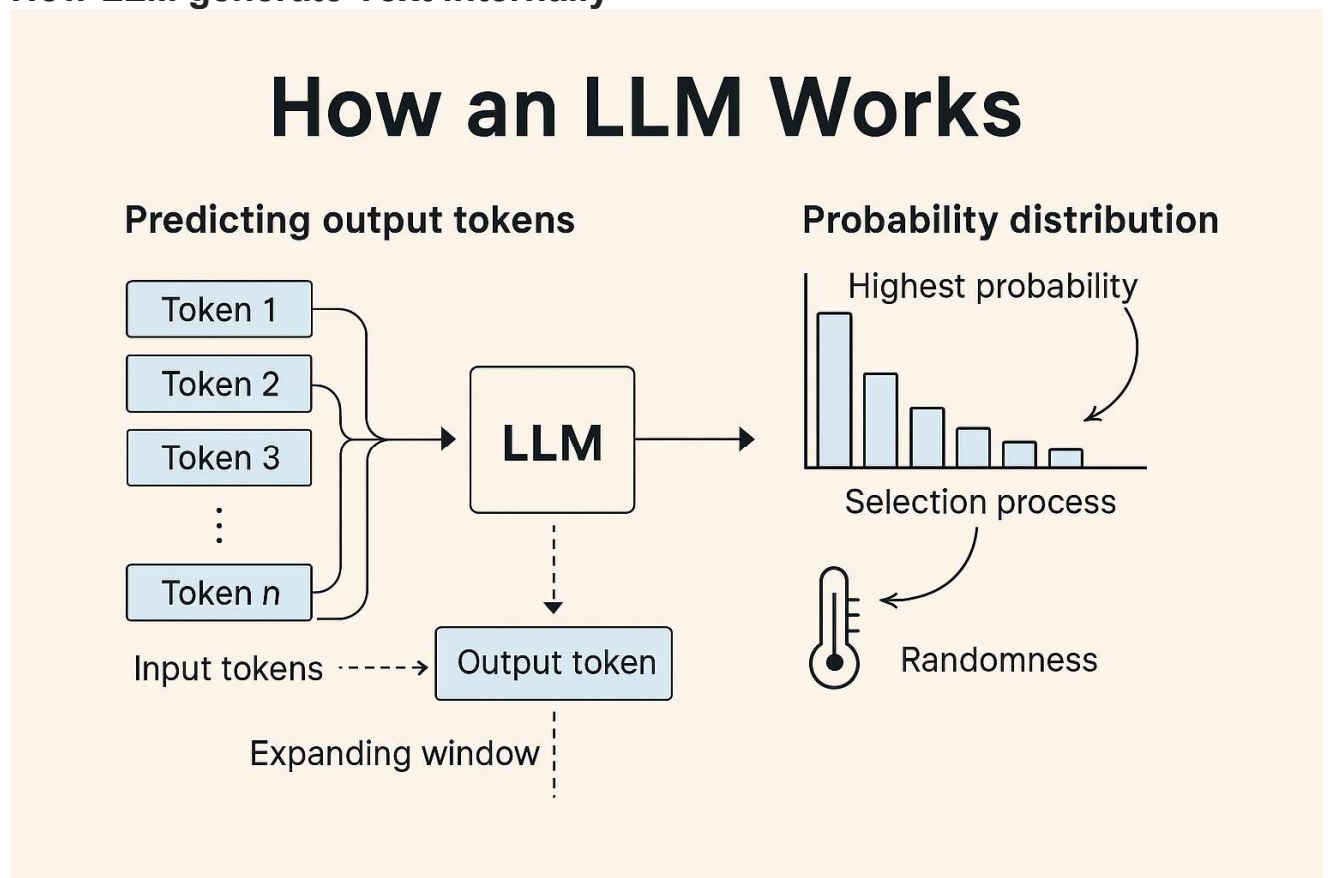
LLM is Large Language Model, it is innovation behind GPT, Gemini, Claude, and LLaMA or similar thing which generate text for us based on questions.

LLM works on **probability, patterns, and numbers** — not human-like understanding.

LLM in simple words is neural network trained on massive text data to predict the next most likely word (token) based on the given input.

LLM has been trained on : Public websites, books, articles, forums, open-source code, and licensed datasets. LLM has trained on huge data , you can say billions of parameters.

How LLM generate Text internally



Suppose User Input is “What is Color of Rose” -

1. LLM does not understand English, everything what ever we ask — it get converted into numerical value we called as **Vectors or Embeddings**

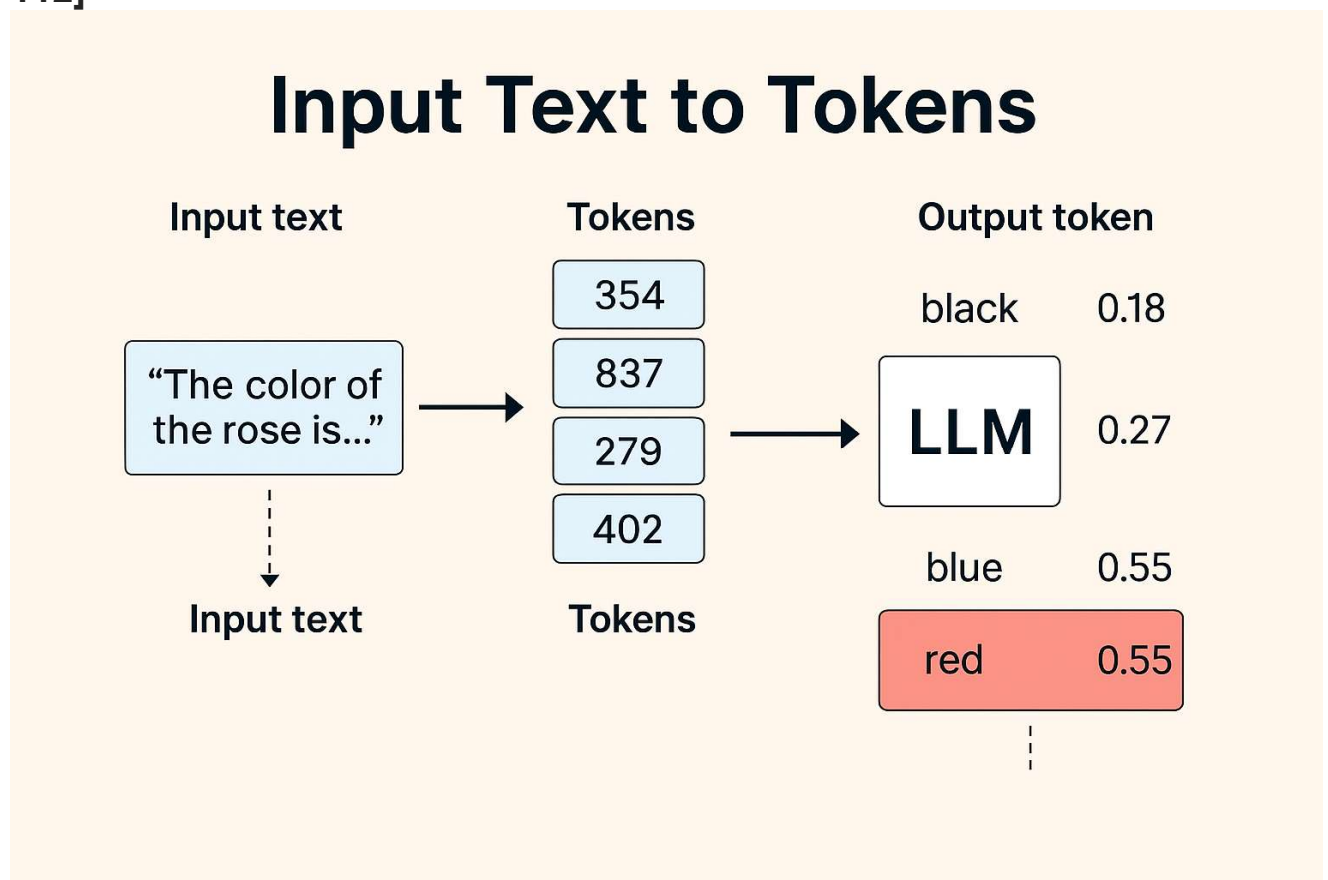
2. “The color of rose is” will be converted into token first then vectors

3. **Tokens** — [“The”, “color”, “of”, “rose”, “is”]

4. **Tokens** → **Embeddings** (Vector Conversion) token converted into Numerical values called embeddings.

5. **Embeddings** of input value will be like below

[“The”, “color”, “of”, “rose”, “is”] — [1012, 3456, 231, 8743, 112]



AI generated image

6. **Probability Calculation** (Next Token Prediction)

Now the model predicts the **next token**, Internally, it calculates probabilities:

like below

Word	Probability
red	0.72
pink	0.15
white	0.08
beautiful	0.02
sweet	0.01

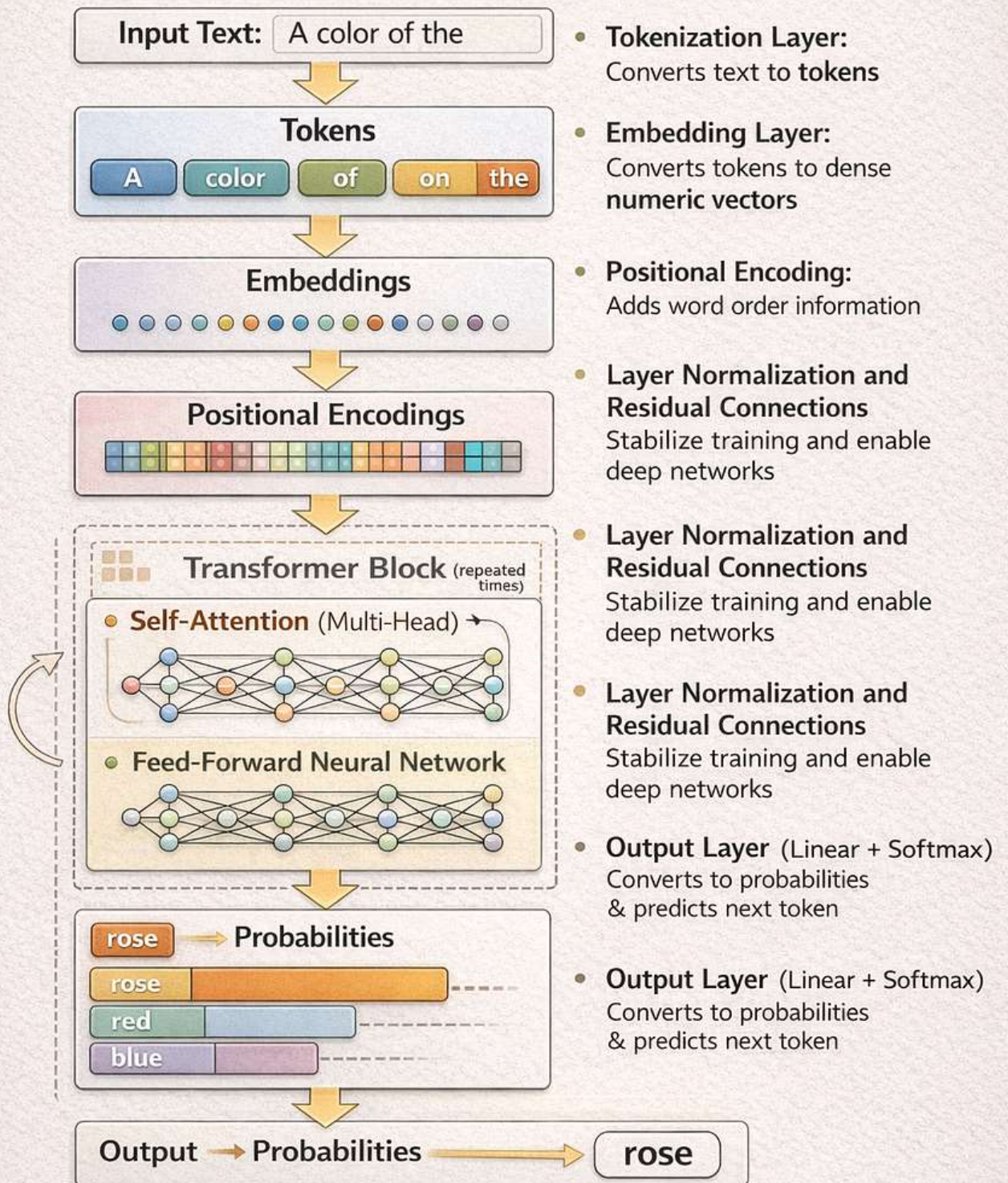
LLM assign probability to next possible tokens as above, red has highest probability and sweet has least so LLM choose token or word which has highest probability

LLM Architecture

Lets understand what is architecture of LLM inside

Most modern LLMs are based on the **Transformer architecture**
Press enter or click to view image in full size

LLM Architecture



Input Text → Tokens → Embeddings → Transformer Blocks → Probabilities
 → Next Token

Input Text → Tokens → Embeddings → Transformer Blocks → Probabilities → Next Token

LLM has below layers

1. Tokenization Layer -> Convert Text into Token
2. Embedding Layer -> Convert Token into Vectors
3. Positional Encoding -> Helps the model understand sequence position
4. Transformer Block (repeated N times) -> Core building unit of an LLM
5. Self-Attention (Multi-Head Attention) -> Determines which words should focus on other words, In our example “Color of rose is” so self attention will be on rose and color only rest of words will be ignored for next word calculation
6. Feed-Forward Neural Network (FFN) -> Enhances feature representation
7. Output Layer -> Converts final vectors into probabilities
8. Result -> LLM choose highest probability word

Conclusion

We have understand high level working of LLM Models, most of model have similar working.

High Level -> Every input convert into vector and then LLM assign probability to next possible word.

As developer we can use these models in application for Text generation, i have created Article on that using google gemini model with C#

Simple Text generation using C#